Independent University, Bangladesh

CSE313 Compiler Construction · Practice Set B · Mid + Quiz · Time: 1h30m

New questions. Q2 needs a conversion before LL(1); quiz Q2 has hidden left recursion.

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following grammar is Ambiguous or NOT. If the grammar is ambiguous, convert it to unambiguous form. **[7]**

```
A → A + A | B
B → B * B | C
C → ( A ) | x
```

2. The grammar below is left recursive. Make it ready for LL(1), then write First() and Follow(), draw the LL(1) Parsing Table and parse `n - ( n )`. (b) Parse `n ( n`. If parsing is not possible, point out the reasons. **[8]**

```
E → E - T | T
T → n | ( E )
```

3. Find True/False: **[5]**

(a) Left factoring removes left recursion.
(b) An ambiguous grammar can be LL(1).
(c) For A → ε, the rule is placed in M[A, b] for every b in FOLLOW(A).
(d) The lexical analyzer produces the parse tree.
(e) In LL(1), the first L means the input is scanned Left to right.

Quiz Question (Total marks: 10)

1. Convert the following Non-Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
A → a b B | a B | c d g | c d e B | c d f B
B → x
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
S → A a | b
A → A c | S d | e
```